Rilevamento e mitigazione di attacchi flooding in SDN

tramite l'entropia degli indirizzi IP di destinazione

Studente: Giuseppe Guarino **Matricola:** DE900002
Repository: https://github.com/giovannistanco/NCIS_unina/tree/main/2026/Guarino_Giuseppe
Insegnamento: Network and Cloud Infrastructures — Prof. Giorgio Ventre — A.A. 2025/2026

1 Obiettivo e scenario

Il progetto realizza un controller Ryu che rileva attacchi di tipo flooding (TCP SYN, UDP, ICMP) diretti verso un host di una rete SDN e li contiene installando automaticamente una regola di drop sul vettore d'attacco, senza interrompere il traffico legittimo e senza dipendere dall'IP sorgente, che un attaccante può falsificare.

Firma dell'attacco. In condizioni normali il traffico che entra in un segmento di rete si distribuisce su più destinazioni; durante un flood la quasi totalità dei pacchetti converge su un unico indirizzo. La grandezza scelta per misurarlo è l'entropia di Shannon della distribuzione degli IP di destinazione su finestre di 5 s: con k destinazioni equiprobabili vale $\log_2 k$ bit, con una sola destinazione tende a 0. La metrica è indipendente dal volume assoluto e dall'indirizzo sorgente, quindi resta valida anche con `--rand-source`.

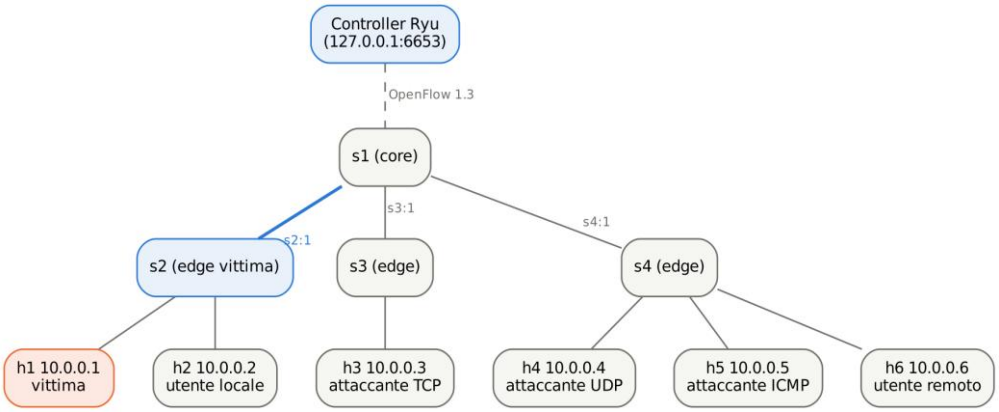


Figura 1: Topologia ad albero a due livelli; la porta 1 di ogni edge switch è la trunk verso il core.

Topologia. La rete (`topology.py`) è emulata in Mininet con switch Open vSwitch in OpenFlow 1.3: un core switch `s1` e tre edge switch. Per l'ordine delle `addLink` la porta 1 di `s2`, `s3` e `s4` è sempre la trunk verso `s1`. I ruoli sono: `h1` vittima, `h2` utente locale, `h6` utente remoto che genera il traffico legittimo (UDP/5001 verso `h1` e `h2`, circa 20 pkt/s per destinazione), `h3`, `h4`, `h5` attaccanti con `hping3` a 1000 pkt/s (10.000 pacchetti) rispettivamente in TCP SYN sulla porta 80, UDP sulla porta 53 e ICMP echo.

2 Il controller

Il controller è un unico modulo Ryu (`controller.py`) basato su un learning switch OpenFlow 1.3. La Tabella 1 riassume strutture dati e metodi aggiunti.

Elemento	Tipo	Ruolo
`datapaths`	dict	switch connessi, aggiornato su EventOFPStateChange (MAIN/DEAD)
`flow_stats_history`	dict	per ogni switch e regola: (packet_count, duration_sec) dell'ultima lettura
`entropy_history`	dict	per ogni switch: ultime 10 entropie di finestre classificate come sane
`trunk_ports`	dict	porta trunk di ogni edge switch {2:1, 3:1, 4:1}; s1 è escluso
`_monitor`	thread	ogni 5 s invia OFPFlowStatsRequest a tutti gli switch
`_flow_stats_reply_handler`	handler	calcola i delta, l'entropia e applica il criterio di decisione
`mitigate_attack`	metodo	sceglie il vettore dominante e installa la regola di drop

Tabella 1: Strutture dati e metodi aggiunti al learning switch.

2.1 Evoluzione dell'approccio di rilevamento

Il rilevatore è stato riprogettato quattro volte; ogni versione nasce da un difetto osservato nei test della precedente. La differenza principale è *da dove* il controller ricava i conteggi (Tabella 2).

Versione	Sorgente dei conteggi	Problema osservato
V1	PacketIn	dopo l'installazione della regola il controller non vede più il flusso
V2	PacketIn + hard_timeout = 1 s	campionamento: il rapporto attacco/legittimo viene compresso
V3	FlowStats su tutti i flussi	le risposte della vittima rendono la distribuzione simmetrica
V4 (finale)	FlowStats, solo in_port = trunk	—

Tabella 2: Versioni del rilevatore.

V1 — entropia sui PacketIn. Il controller incrementava `num_ip_dst` a ogni `EventOFPPacketIn` e chiudeva la finestra al primo PacketIn successivo ai 5 s. Il difetto è strutturale: dopo il primo pacchetto il controller installa una regola di priorità 1 e i pacchetti successivi vengono inoltrati dal data plane. Il controller osserva quindi 1–3 pacchetti per flusso indipendentemente dal rate: un flood a 1000 pkt/s e un ping a 1 pkt/s producono lo stesso numero di eventi e la soglia `MIN_PKT = 50` non viene mai raggiunta. A questo si aggiungevano contatori globali (un pacchetto che attraversa `s3→s1→s2` genera tre PacketIn ed è contato tre volte) e una finestra che, a rete silenziosa, non veniva mai chiusa.

V2 — campionamento forzato. Per costringere il traffico a risalire al controller la regola L2 veniva installata con `hard_timeout = 1`. A ogni scadenza raggiungono il controller solo i pacchetti che arrivano prima della reinstallazione, circa *rate × latenza del controller*: un ping a 1 pkt/s è contato per intero, un flood a 1000 pkt/s produce poche unità. Il rapporto attacco/legittimo passa da circa 1000:1 a pochi a uno e l'entropia perde capacità discriminante; `MIN_PKT` era stato ridotto a 5, rendendo le decisioni dominate dal rumore.

V3 — polling dei contatori. Il PacketIn viene usato solo per installare regole con match fine (porta d'ingresso, MAC, `ipv4_dst`, `ip_proto`, porta L4) e `idle_timeout = 20`; un thread legge ogni 5 s i contatori con `OFPFlowStatsRequest` e calcola i pacchetti per finestra. Il conteggio è ora completo, ma emerge un nuovo problema: lo switch conta entrambe le direzioni. A un SYN su porta chiusa la vittima risponde con un RST, a un echo request con un echo reply, quindi la distribuzione diventa circa {`h1`: N, attaccante: N} con H ≈ 1 bit, lo stesso valore di un ping legittimo. Con soglia statica pari a 1,0 il rilevamento dipendeva da un singolo pacchetto (9990 SYN e 9985 RST danno H = 0,99999995), mentre un trasferimento TCP legittimo (N segmenti, N/2 ACK) produceva H = 0,918 e un falso positivo. La metrica misurava la simmetria del traffico, non la sua concentrazione.

V4 — polling ristretto alla trunk (versione finale). Su ogni edge switch vengono contate solo le regole con `in_port` uguale alla porta trunk, cioè le richieste che arrivano dal core verso gli host locali. Le risposte generate dagli host locali entrano da porte host e vengono escluse; ogni pacchetto è contato una sola volta, all'ingresso dell'edge di destinazione, e il core `s1` non partecipa al rilevamento. Su `s2` un attacco distribuito proveniente da più edge converge comunque su `h1` e abbassa l'entropia. La versione introduce inoltre l'apprendimento della baseline e la gestione del reset dei contatori descritti in 2.2.

2.2 Calcolo dell'entropia e criterio di decisione

Per ogni regola *f* con priorità 1 si memorizzano il contatore *c* e la durata *d*. Il numero di pacchetti nella finestra è la differenza tra due letture; se il contatore o la durata diminuiscono, la regola è scaduta ed è stata ricreata, quindi il delta coincide con il nuovo contatore:

$$\Delta_f(t) = c_f(t) - c_f(t-1) \quad \text{se} \quad c_f(t) \geq c_f(t-1) \;\wedge\; d_f(t) \geq d_f(t-1), \quad \text{altrimenti} \quad \Delta_f(t) = c_f(t)$$

Aggregando le regole con `in_port = trunk` per IP di destinazione *i* si ottengono N_i, il totale N e le probabilità $p_i = N_i / N$:

$$H = -\sum_i p_i \log_2 p_i \qquad \theta = 0{,}7 \cdot \bar{H}_S \;\; \text{(media delle entropie in } S\text{)}$$

dove S è l'insieme delle ultime 10 finestre sane. Una finestra è classificata come attacco se $N > 50$, $|S| \geq 5$ e $H < \theta$; altrimenti entra in S solo se $N > 50$ e $H \geq 0{,}5$. Il vincolo su $|S|$ impedisce decisioni prima che la baseline sia nota; il filtro $H \geq 0{,}5$ impedisce che un attacco avvenuto durante l'apprendimento entri nello storico e porti la soglia a zero (fenomeno osservato nei test: dopo cinque finestre d'attacco la media era 0 e la condizione $H < 0$ non poteva più essere vera). Con il warm-up di `h6` verso due destinazioni la baseline vale 1 bit e θ = 0,70; durante il flood la vittima riceve oltre il 95% dei pacchetti e H scende sotto 0,3.

```
if entropy < din_threshold and current_tot > self.MIN_PKT and ha_baseline_sufficiente:
    ip_vittima = max(current_num_ip, key=current_num_ip.get)
    self.mitigate_attack(msg.datapath, ip_vittima, current_vectors)
elif current_tot > self.MIN_PKT and entropy >= self.DEFAULT_ENTROPY_THRESHOLD:
    self.entropy_history[dpid].append(entropy)      # solo finestre sane
```

2.3 Regole OpenFlow e conteggio affidabile

Prio	Match	Azione	Timeout	Origine
0	*	controller	—	SwitchFeatures
1	in_port, eth_src, eth_dst, eth_type=0x0800, ipv4_dst, ip_proto, [tcp_dst|udp_dst]	output	idle 20 s	PacketIn IPv4
1	in_port, eth_src, eth_dst, eth_type	output	idle 20 s	PacketIn non IPv4
100	in_port, eth_type, ipv4_dst, ip_proto, [tcp_dst|udp_dst]	drop	hard 30 s	mitigate_attack

Tabella 3: Regole installate dal controller.

Il conteggio è affidabile solo se ogni flusso IPv4 ottiene la propria regola specifica. In una versione intermedia la regola per i pacchetti non IPv4 non conteneva `eth_type`: le regole create dagli scambi ARP unicast del `pingAll` (ad esempio `h6→h1`) corrispondevano a tutto il traffico tra i due MAC e, avendo la stessa priorità della regola IPv4, inoltravano anche l'UDP del warm-up. Il traffico non risaliva più al controller, la regola con `ipv4_dst` non veniva creata e l'handler delle statistiche, che richiede `ipv4_dst` nel match, non vedeva nulla: la baseline restava in apprendimento. La diagnosi è stata fatta confrontando `tcpdump` su `h6-eth0` (pacchetti UDP/5001 in uscita a 20 pkt/s per destinazione) con `dump-flows s4` (nessuna regola su 5001); con traffico continuo l'`idle_timeout` non scadeva mai. Aggiungere `eth_type` al match limita la regola al solo protocollo che l'ha generata.

2.4 Mitigazione

Tra i flussi diretti alla vittima viene scelto il vettore con più pacchetti nella finestra, cioè la tupla (`in_port`, `ip_proto`, porta L4), e sull'edge della vittima si installa una regola di priorità 100 senza azioni (drop) con `hard_timeout = 30`. Per TCP e UDP il blocco è L4 (ad esempio `in_port=1, tcp_dst=80, ipv4_dst=10.0.0.1`), per ICMP è L3 su `ip_proto=1`. Il match usa la porta fisica d'ingresso e non l'IP sorgente, per cui il blocco resta valido con IP falsificati, mentre il traffico legittimo verso altre porte L4 (UDP/5001) continua a essere inoltrato. I pacchetti scartati vengono contati dalla regola di drop, che il monitoraggio ignora (priorità ≠ 1).

3 Risultati sperimentali

Sono state eseguite due sessioni complete con `topology.py --test all`, la prima con IP reali e la seconda con `--spoofing`. Ogni sessione prevede 40 s di warm-up e i tre attacchi in sequenza, separati da 32 s per la scadenza della regola di drop. La Figura 2 è ricavata dai file CSV salvati dal controller e riporta, per `s2`, entropia e pacchetti entranti dalla trunk per finestra.

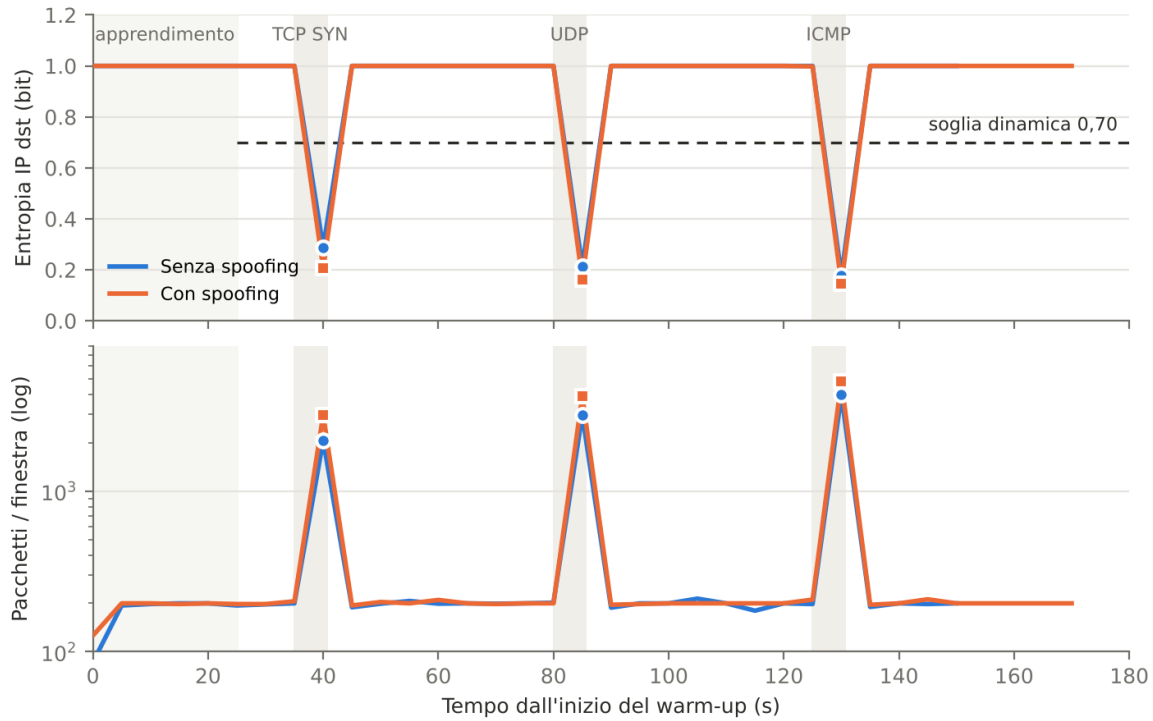


Figura 2: Entropia e pacchetti per finestra su s2 nelle due sessioni; i marcatori indicano le finestre classificate come attacco.

Attacco	N (IP reali)	H (IP reali)	N (spoofing)	H (spoofing)
TCP SYN /80	2062	0,288	2978	0,207
UDP /53	2982	0,212	3908	0,162
ICMP echo	3964	0,179	4823	0,146
Finestre normali	180–214	1,000	194–212	0,998–1,000

Tabella 4: Finestre di attacco su s2 ($\theta = 0{,}70$) e intervallo delle finestre normali.

Estratto del CSV della prima sessione intorno al TCP SYN flood:

```
Timestamp,Switch_DPID,Pacchetti_In_Trunk,Entropia,Soglia,Attacco
16:42:57,2,200,1.0,0.7,False
16:43:02,2,2062,0.2882,0.7,True
16:43:07,2,189,1.0,0.7,False
```

- **Latenza di rilevamento.** Tutti e sei gli attacchi sono stati classificati nella prima finestra di polling che li conteneva, quindi entro 5 s dall'inizio. La soglia dinamica è attiva dopo 25 s di warm-up (cinque finestre sane).
- **Nessun falso positivo.** Su 66 finestre di `s2` nelle due sessioni, le sole classificate come attacco sono le 6 finestre di flood; il traffico normale ha sempre avuto H tra 0,998 e 1.
- **Efficacia del drop.** Nella finestra successiva al rilevamento N torna a circa 200 e H a 1. Il valore è coerente con il solo traffico di `h6` (2 destinazioni × 20 pkt/s × 5 s), che quindi non viene interrotto, e il flood non viene più contato perché finisce nella regola di priorità 100. Le finestre d'attacco contengono meno dei 5000 pacchetti teorici perché il flood inizia a finestra già avviata e viene bloccato alla lettura successiva.
- **Spoofing.** Con `--rand-source` il comportamento è identico: né il conteggio né il match di drop usano l'IP sorgente.

Nei CSV compaiono anche righe di `s3` e `s4` con circa 10 pacchetti e H = 0: sono gli ICMP Port Unreachable che `h1` e `h2` inviano a `h6` in risposta all'UDP/5001. Restano sotto `MIN_PKT` e non producono decisioni.

4 Riproduzione dell'esperimento

Ambiente. I test sono stati eseguiti con Ubuntu 22.04.5 LTS, Python 3.10, Mininet 2.3.1b4, Open vSwitch 2.17.12, Ryu 4.34, OpenFlow 1.3 e hping3.

Avvio. Il controller va lanciato prima della rete, in due terminali distinti:

```
# terminale 1
ryu-manager controller.py
# terminale 2
sudo mn -c && sudo python3 topology.py --test all     # oppure --test tcp udp, --spoofing
```

Senza `--test` la rete parte in modalità interattiva. In modalità test lo script esegue `pingAll` per popolare le tabelle ARP e il MAC learning, configura `h1` (`icmp_echo_ignore_all=1` e drop dei RST in uscita con `iptables`) per ridurre il traffico di ritorno, avvia il warm-up da `h6`, attende 40 s e lancia gli attacchi richiesti. Dopo 10 s di warm-up stampa i processi `hping3` attivi e le regole su UDP/5001 di `s2`, per verificare che la baseline venga costruita.

Risultati osservabili. (1) Nel log del controller, finestre di `Switch 2` con circa 200 pacchetti e H ≈ 1, poi la soglia passa da `Apprendimento` a 0,7. (2) A ogni attacco una riga `ATTACCO DOS RILEVATO` seguita dalla riga di mitigazione L4 o L3. (3) Con `sh ovs-ofctl -O OpenFlow13 dump-flows s2 | grep priority=100` la regola di drop con `n_packets` crescente, mentre la regola `udp_dst=5001` continua a contare. (4) Il file `metriche_ddos_<timestamp>.csv` con una riga per switch e finestra.

Nota metodologica. Il warm-up deve precedere gli attacchi: senza almeno cinque finestre sane il controller non prende decisioni. Gli attacchi sono separati da 32 s perché la regola di drop resti attiva per tutta la durata del flood e scada prima del successivo.

5 Limiti noti e sviluppi

- **Concentrazione legittima.** Un flusso legittimo intenso verso un solo host abbassa l'entropia come un attacco: la metrica rileva la concentrazione del traffico, non la sua natura.
- **Granularità del blocco.** Il drop è installato sulla trunk dell'edge della vittima, dove tutti gli host remoti condividono la stessa `in_port`: un blocco ICMP colpirebbe anche ping legittimi remoti verso `h1` e il flood continua a occupare il core. Uno sviluppo naturale è localizzare la porta host dell'attaccante con le statistiche di `s3`/`s4` e installare il drop alla sorgente.
- **Un vettore per finestra.** Con più attaccanti simultanei servono più finestre per bloccarli tutti; la regola viene inoltre reinviata a ogni finestra finché l'attacco prosegue.
- **Configurazione statica.** La mappa `trunk_ports` dipende dall'ordine delle `addLink`; potrebbe essere ricavata con la scoperta dei link di Ryu (`--observe-links`). Al posto del drop si potrebbero usare i meter di OpenFlow 1.3 per limitare il rate, riducendo l'impatto di un falso positivo.